Morpheus HypnoLigh Project

Objective

The objective of this project is to create a device for visual and auditory stimulation. This type of device is often referred to as FLS (Flicker Light Stimulation) or AVS (Audio-Visual Stimulation). Flicker Light Stimulation (FLS) is a non-invasive technique that uses rhythmic light variation to influence brain activity and the state of consciousness. It involves emitting rhythmic light pulses at specific frequencies, typically ranging from 3 Hz to 50 Hz. These light pulses stimulate the visual system, leading to a phenomenon called "visual synchronization," in which brain activity synchronizes with the frequency of the light. This synchronization can induce altered states of consciousness, like those experienced during deep meditation or under the influence of psychedelics, often accompanied by striking visual patterns and sensations beyond conscious control.

The main components of the machine are:

- At the front, an aluminum printed circuit board equipped with cold white light-emitting diodes
- Behind it is a second printed circuit board that contains the control electronics. In particular an ESP32-S3 microcontroller and the LED driver circuits
- A 120x120 PWM cooling fan;
- The entire assembly is housed in a flat, compact enclosure—making it easy to transport—and features a microphone stand-style mounting system

Features

Non-exhaustive list of features

- Modes of operation
  - Session local: standalone mode where a session is loaded from local memory and controlled locally
  - Session remote: the session selection and control are done remotely using BLE or Wifi. In this mode the remote-control program can drive multiple machines.
  - Real-time local: the machine is controlled in real time locally (rotary knobs, mouse, Gampad, ...)
  - Real-Time remote: the machine is controlled remotely using BLE or Wifi. In this mode the remote-control program can drive multiple machines.
- Bluetooth Low Energy connectivity (must have). Used to connect to control program, Sound system, peripherals (mouse, Gamepad, ...)
- 8 groups of white LEDs (must have)
- Signal generators with sine, triangle, square waveforms (must have)
- It must be possible to pause and seek loaded sequences (must have)
- Input devices: Parameter controller (rotary koob & switches), joystick, mouse ... (should have). Support for external controllers (e.g. midi controller)
- Output devices Sound output – BLE/Analog (could have)
- Wifi-Web server interface (could have)
- Low-cost hardware – less than 100$ (must have)
- Low-noise controlled fan

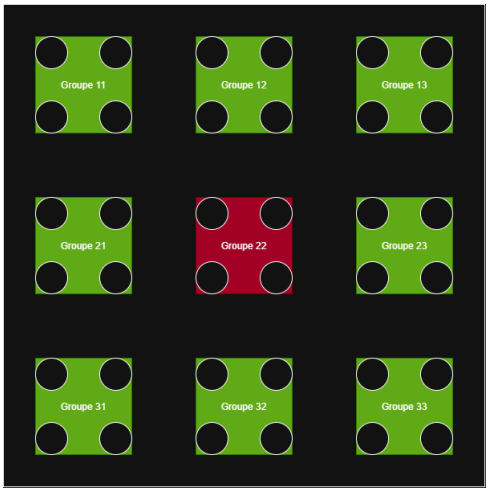
- Compact and easy to carry with tripod mounting threads
- Store sequences locally on SD card
- Capacitive Touch Display (can be a dev board)

PCB des LED

This PCB is unique because it uses an aluminum substrate to dissipate the heat generated by the LEDs. In addition to the LEDs, it contains temperature sensors that are used to automatically activate the fan in the event of overheating. This PCB is connected to the control electronics via two flat cable connectors.

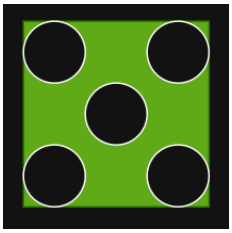
It is important to note that, based on numerous experiments, the position of the LEDs on the PCB has absolutely no effect on the user's perception. For example, a diode lighting up in the upper left corner or in the lower right corner is perceived exactly the same way by the user.

Consequently, it was decided to arrange the LEDs into 8 groups of 4 as follows



We are using cold white LED with a power rating of approximately one watt, so each group can provide a maximum power of 4 watts, and all 8 groups together provide a total power of 32 watts.

If this power is insufficient, the groups of 4 LEDs can be replaced with groups of 5 LEDs, which would increase the total power to 40 watts.

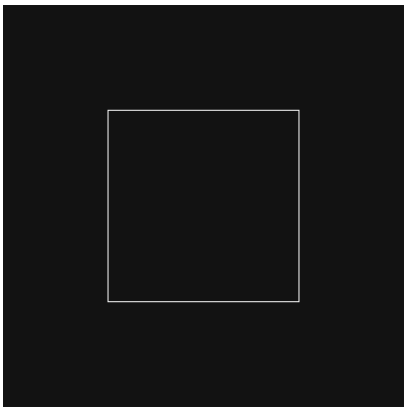


You will notice in the diagram that there is a 9th group (Group 22). The use of this group is still under consideration, as it would require additional LED drivers. The LEDs would be either RGBW or warm white. This would bring the total power to 36 watts (or 45 watts if using groups of 5).

As previously mentioned, this PCB is also equipped with ***one or several*** temperature sensors distributed among the groups and ***2*** connectors for ribbon cable to connect ***the board*** to the control electronics.

Control PCB

The control electronics will be placed on a standard PCB drilled in its center with a square of about 90 by 90 cm.



The purpose of this hole is to allow air from the fan, located below, to circulate and cool the LED PCB. This means that the electronics will need to be distributed around the perimeter of this PCB. As previously mentioned, the key components of this PCB will be an ESP32-S3 microcontroller and 8 ***led driver*** circuits for the 8 groups of LEDs.

Driving a group of led

After examining several LED drivers, the choice fell on the AL8862 circuit, whose characteristics are perfectly suited to the use of the machine. The control of this circuit can be done either in an analog or digital way.

The analog control of the AL8862 has limitations:

- **Limited range** : only 0.5V to 2.5V
- **Potential nonlinearity**
- **Additional Complexity** with D/A Converters

For these reasons we will not use analog control and the settings will be made by PWM modulation.

A two-level **PWM modulation** technique is used that would control both the brightness and the flashing frequency of your LED.

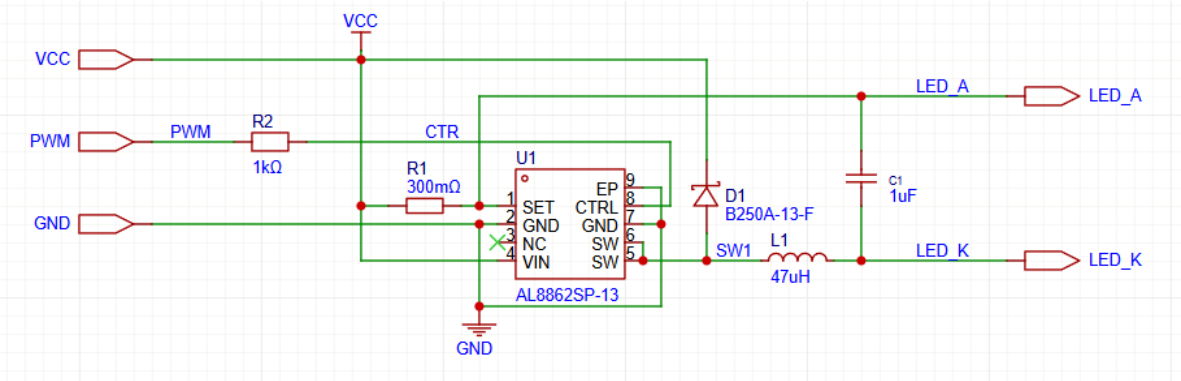
We use a:

- **Carrier signal** : High-frequency PWM (500Hz-1kHz) to control brightness
- **Modulating Signal** : Low Frequency PWM (1-100Hz) to Create Visible Flashing Effect

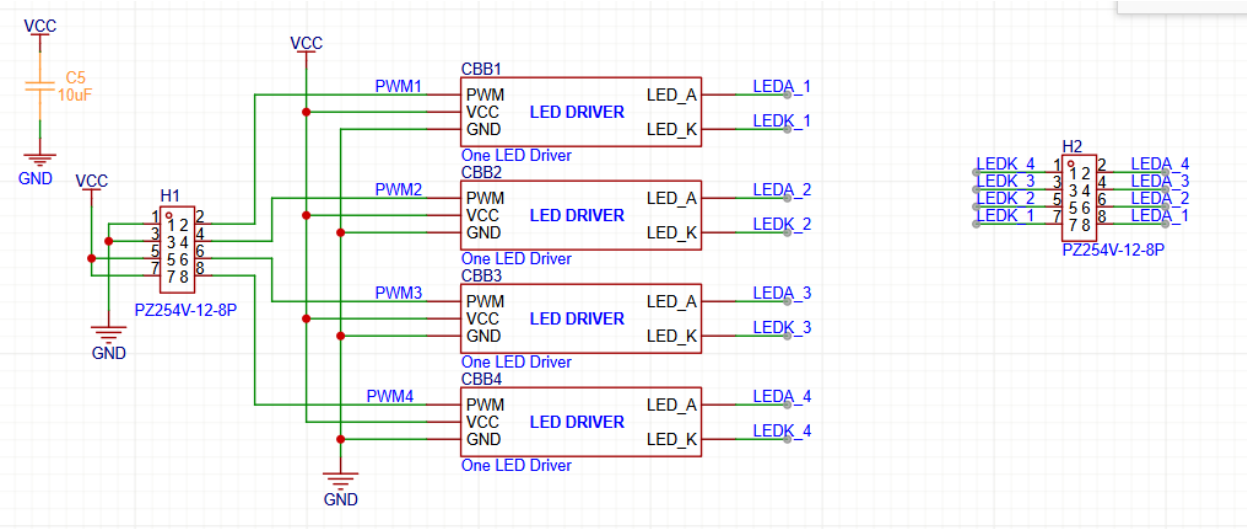
The following capabilities of the ESP32 are used to control the signals:

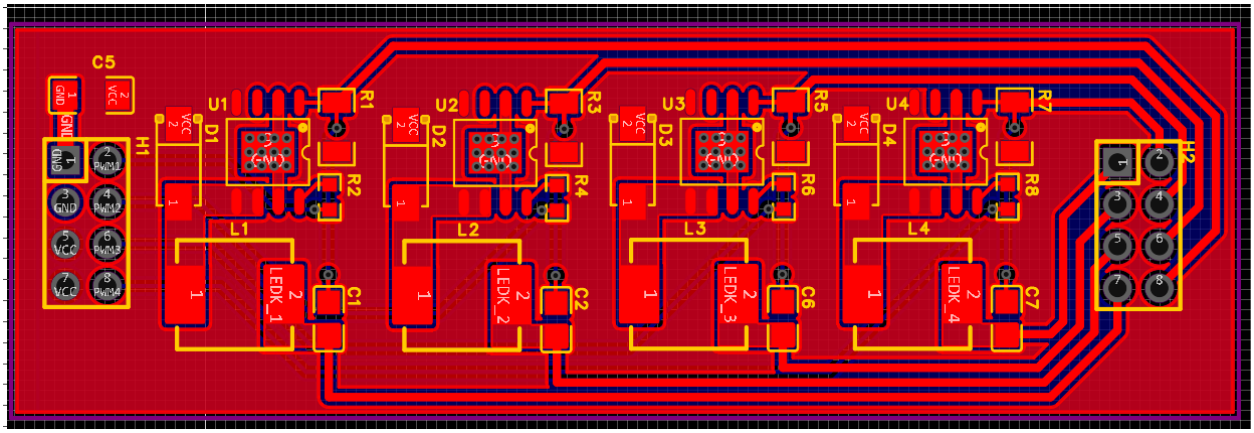
- 16 independent PWM channels via LEDC device
- Frequencies : from 1Hz to 40MHz
- Resolution : 1 to 16 bits
- Flexibility : each channel can have its own frequency and duty cycle

The control circuit of a group of LEDs and therefore the following:



Note that the validation prototype can be built using interconnected test modules: one ESP32-S3 module and two Led driver modules, each containing driver for four LED.





Generators / Modules

For the various types of generator modules used to drive the LEDs, I drew inspiration from the modules in my Korg MS20/MS50/SQ10 synthesizers from the 1980s.

The basic modules are as follows:

- A sequencer that defines the various parameters of the modules for each step of a sequence
- 8 ASR module (2 per oscillators): They deliver a signal composed of an attack time, a sustain time, and a release time. They are used to drive the duty cycle and the brightness of the main oscillators.
- 4 low frequency oscillators (one per main oscillator): They can deliver sinusoidal, triangle, or square waveforms to modulate the frequency of the main oscillators.
- 4 main oscillators that drive the led group. They can deliver sinusoidal, triangle, or square waveforms. They have four inputs (frequency, modulation, duty cycle, brightness) driven by the modules described above. Potentially they may also have a phase input to change the phase of the signal produced by the main oscillator.

L'Implémentation dans l'ESP32 de la modulation PMW

L'implémentation du code pour commander des différents groupes de leds ressemblera à cela :

```
// Paramètres
const int LED_PIN = 2;
const int PWM_FREQ = 1000;         // Porteuse 1kHz
const int PWM_RESOLUTION = 10;      // 10-bit = 1024 niveaux
const int PWM_CHANNEL = 0;

// Variables pour l'enveloppe basse fréquence
float envelope_freq = 10.0;        // 10Hz clignotement
int brightness_max = 512;          // 50% luminosité max (0-1023)

void setup() {
  ledcSetup(PWM_CHANNEL, PWM_FREQ, PWM_RESOLUTION);
  ledcAttachPin(LED_PIN, PWM_CHANNEL);
}

void loop() {
  // Calcul de l'enveloppe sinusoïdale ou carrée
  float time_s = millis() / 1000.0;
  float envelope = (sin(2 * PI * envelope_freq * time_s) + 1) / 2;

  // Ou pour une enveloppe carrée :
  // float envelope = (sin(2 * PI * envelope_freq * time_s) > 0) ? 1.0 : 0.0;

  int pwm_value = envelope * brightness_max;
  ledcWrite(PWM_CHANNEL, pwm_value);

  delay(1); // Actualisation 1ms
}
```

Code optimisé pour formes d'onde variées

```
enum WaveForm { SQUARE, SINE, TRIANGLE, SAWTOOTH };

class LEDController {
private:
  int pin, channel;
  float carrier_freq, envelope_freq;
  int max_brightness;
```

```
  WaveForm waveform;

public:
  void setEnvelope(float freq, WaveForm wave, int brightness) {
    envelope_freq = freq;
    waveform = wave;
    max_brightness = brightness;
  }

  void update() {
    float t = millis() / 1000.0;
    float envelope = calculateEnvelope(t);
    int pwm_val = envelope * max_brightness;
    ledcWrite(channel, pwm_val);
  }

private:
  float calculateEnvelope(float t) {
    float phase = 2 * PI * envelope_freq * t;
    switch(waveform) {
      case SQUARE: return (sin(phase) > 0) ? 1.0 : 0.0;
      case SINE: return (sin(phase) + 1) / 2;
      case TRIANGLE: return 2 * abs(fmod(phase/(2*PI), 1.0) - 0.5);
      // etc.
    }
  }
};
```

Ventilateur de refroidissement

A 120 mm x 120 mm fan will be located on the back of the control PCB. This fan must be a low-noise model (such as a Corsair model) with PWM control according to the temperature of the LED PCB.

Housing

The housing will be 3D-printed. It will likely consist of three parts: a perforated back plate to allow fresh air to enter; a central section to which the LED PCB will be attached; and holes along the edge of this section to allow air to escape. Finally, there will be a top section.

The fan will be secured with four screws to the control PCB. The LED PCB will be attached to the control PCB using four or eight standoffs.

Alimentation

L'alimentation de la lampe se fera par un bloc situé à l'extérieur afin de minimiser le dégagement de chaleur à l'intérieur de la lampe. En fonction du nombre de l'aide utilisée par groupes il faudra probablement prévoir une alimentation avec une sortie en courant continue d'une vingtaine de volts et d'environ 60 ampères

References

- Arduino | Adafruit I2C Quad Rotary Encoder Breakout | Adafruit Learning System